Topic 12: Templates - Function and Class Templates

12.1 Why Templates?

- Without templates, you'd write separate swap functions for int, double, string, etc.
- Templates let you write code once and have the compiler generate type-specific versions automatically
- This is called generic programming -- the algorithm is independent of the data type
- The C++ Standard Library (STL) is built entirely on templates

12.2 Function Templates

- Syntax: template <typename T> ReturnType funcName(T param) { }
- typename and class are interchangeable in template parameter lists
- The compiler deduces T from the argument type at the call site
- You can explicitly specify: swap<double>(a, b);

```
template <typename T>
T maxOf(T a, T b) {
    return (a > b) ? a : b;
}

template <typename T>
void mySwap(T& a, T& b) {
    T temp = a; a = b; b = temp;
}

int main() {
    cout << maxOf(3, 7) << endl;       // T = int
    cout << maxOf(3.5, 2.1) << endl;   // T = double
    string s1 = "hi", s2 = "bye";
    mySwap(s1, s2);
}
```

12.3 Class Templates

- Define a generic class where the type is a parameter
- Instantiate with a specific type: Stack<int>, Stack<string>
- All member function definitions outside the class must also carry the template header

```
template <typename T>
class Stack {
    T   data[100];
    int top = -1;
public:
    void push(T val) { data[++top] = val; }
    T    pop()       { return data[top--]; }
    bool empty()     { return top == -1; }
    T    peek()      { return data[top]; }
};

int main() {
    Stack<int>    intStack;
    Stack<string> strStack;
    intStack.push(10);
    strStack.push("hello");
}
```

12.4 Template Specialisation

- Provide a custom implementation for a specific type when the generic version won't work correctly
- Full specialisation: template <> ReturnType funcName<SpecificType>(params) { }
- Example: printing a bool as 'true'/'false' instead of 1/0
